

Decision Models at the Game Table: Where Non-Generative Classifiers Help and Fail in LLM Tooling for Tabletop Role-Playing Campaigns

Kostadis Roussos
Independent researcher
kostadis@gmail.com

October 2026

Abstract

Decision models are a recent class of non-generative language models: a backbone reads a *state* once and a small head returns a probability for every option of every typed question (multiple choice, yes/no, ordinal score), with no text generated. We test six of them: TypeSafe’s hosted Jev, which introduced the SystemOne interface, and two open families released in October 2026, Cloudflare’s Clef and vLLM Semantic Router’s Decision 2.0, served on a local NVIDIA DGX Spark (GB10). We ask whether they can replace or assist a generative model in the judgment steps of a working LLM pipeline for tabletop role-playing campaigns. Labels are the game master’s own past rulings, frozen into one versioned dataset: 766 reviewed transcript-correction cards, 864 triaged tokens, 497 spelling rulings and 115 entity-type merge decisions, plus 12 paired NPC-behaviour scenarios. Decision models work where the label is a coarse property readable from meaning: Clef 27B and Jev each forwarded about half of a transcript’s unknown-token pile on a held-out split without discarding any of 84 real names, and five of six models moved NPC behaviour in the expected direction on 11 of 12 paired scenarios against 5 of 12 for a published dice-table system. They fall short where the label is a string-level judgment (choosing a canonical spelling: 66–69% for the best decision models against 76% for a local generative model), a house convention, or a consistency property across many items (typing extracted facts one at a time created about as many split entities as it repaired; a model-free per-entity vote matched the GM’s filing 32 times in 33 where it decided). Jev is the exception that shows what to ask for: on spelling it abstained instead of guessing, with 28 confident wrong names against 109 for Clef 27B. Fine-tuning a 4B decision model on the GM’s rulings recalibrated it (triage AUC 0.90 to 0.96) but did not teach the conventions. We also report that an apparently strong verdict-prediction result was an artifact of the classifier reading the upstream model’s own hedging, and that vendor latency claims did not transfer to the GB10 (127 ms measured vs. 18.4 ms claimed).

1 Introduction

Large language model pipelines for long-running creative work make many small judgment calls: is this token a misheard proper noun, which entity does this fact describe, how would this character react. In a pipeline that turns recorded game sessions into campaign records, each of these calls feeds the next step, and an error compounds silently unless a human checkpoint catches it. A practitioner therefore wants two things from the model making those calls: it should be right, and it should *know when it is not*, so that uncertain cases can be routed to the human instead of being decided.

Decision models address both wants directly. Instead of generating an answer, they score a fixed set of options and return a probability for each, in one forward pass. TypeSafe’s *Jev* [1] is a hosted model behind the *SystemOne* request format. Two open families exposing the same format were released within days of each other in October 2026: Cloudflare’s *Clef* (27B and 9B backbones) [2] and vLLM Semantic Router’s *Decision 2.0* (0.6B to 27B) [3], both advertising low latency.

This paper reports a practitioner’s evaluation of these models against real workloads rather than benchmarks. The workloads come from an open-source campaign-management toolkit used to run several tabletop role-playing campaigns; the labels come from one game master’s (GM’s) recorded rulings on earlier model output. The open models ran on local hardware (two NVIDIA DGX Spark systems, GB10, 128 GB unified memory each), deliberately: part of the aim was to calibrate what local serving of such models is like in practice. *Jev*, whose weights are not released, was called through its hosted API as a reference point.

We ask four questions:

RQ1 Can a decision model replace a generative model in the *choice* steps of the pipeline (spelling selection, entity typing)?

RQ2 Can it act as a cheap *filter* or *flag* in front of an expensive model or a human reviewer?

RQ3 Are its probabilities useful for *sampling* plausible behaviour (NPC reactions), where a distribution matters more than an argmax?

RQ4 Do the published speed claims hold on this hardware?

Contributions. (i) Results on five tasks for six decision models and a generative baseline, with labels taken from a GM’s own rulings and frozen into a versioned dataset, including two strong negative results; (ii) a characterization of *which kinds of label* decision models can and cannot learn from the input, and of the failure mode (abstain or guess) that separates otherwise similar models; (iii) two methodological pitfalls that produced convincing but spurious results in this study (label leakage through upstream prose; selection bias in a repair-only test set); (iv) a fine-tuning test showing that the convention failure survives training on the GM’s own rulings; (v) deployment notes for the GB10, including a unified-memory accounting bug in an upstream runtime.

2 Background

2.1 Decision models and the SystemOne interface

A request is a **state** (text) and a dictionary of named **questions**, each typed **choice** (a dictionary of option keys to descriptions), **noul** (yes/no) or **score** (an ordered list of levels). The model prefills the state and the question text once; a small head scores every option of every question jointly. The response carries a probability per option. TypeSafe defined the interface (`POST /v1/systemone`) for *Jev*, which it trains for calibrated decisions rather than generated text and serves only as a hosted API [1]. *Clef*’s release implements the head as a separate set of weights (“joint schema head”) on Qwen-family backbones [2]. *Decision 2.0* is served by a standalone runtime in vLLM Semantic Router [5, 6] that exposes `POST /v1/decisions`, a superset of *SystemOne*, with `/v1/systemone` as an alias [4]. Because all three share the interface, every experiment here ran against any model by changing only the endpoint.

2.2 The workloads

The toolkit’s pipeline turns a session’s audio transcript into a reviewed transcript, a structured summary, scene extractions and finally narrative prose. Three steps involve classification-shaped judgments:

- **Transcript spelling pass.** Automatic speech recognition mangles fantasy names (*Zalthir* → “Zalfir”). A pass collects unknown capitalised tokens, filters obvious non-names, proposes a canonical spelling for the rest, and asks the GM to approve each proposal on a review page (Approve / Reject / Discuss).
- **Ensemble fact extraction.** Five extraction passes over each chapter emit facts with a **type** (npc, monster, faction, location, object, event, thread, date). Downstream, facts are grouped into per-entity dossiers keyed by (type, canonical subject).
- **NPC behaviour at the table.** During play the GM must decide how a non-player character responds. A published system, FlexAI [7], does this with d100 tables keyed on the creature’s role and stance; it never sees the situation itself.

2.3 The human-checkpoint principle

The toolkit’s design rule is that a model may draft, but *precision decisions* (scope, ordering, attribution, canonical spellings) require a human checkpoint before they feed another model. This shapes what counts as a useful classifier: a model that cannot be trusted to decide may still be valuable if it reliably *orders* or *flags* work for the human, provided nothing is dropped silently.

3 Setup

3.1 Models and serving

Table 1 lists the models. Clef and clef-flash were served by a thin FastAPI wrapper around the release’s own model code using PyTorch 2.11 (CUDA 13.0); the release’s suggested vLLM command would load only the chat backbone and silently drop the decision head. Decision 2.0 was served by `vllm-sr-runtime` [4] at a pinned commit, one model per container. Jev was called as `jev-1.13.0`, pinned rather than the moving `jev-latest` alias, through TypeSafe’s public API from the same workstation that drove the local runs. The generative baseline was `qwen3.8-flash-next` (NVFP4/FP8 hybrid quantisation, served by vLLM) with reasoning either disabled (“fast”) or enabled with a 4,000-token budget (“thinking”). Questions, option descriptions and prompts are identical across models.

3.2 Labels and the frozen dataset

All labels are rulings the GM made on earlier model output in one campaign (an adapted *Out of the Abyss*). Four sources are used:

1. **766 transcript-correction review cards** across 59 sessions: 662 approved, 98 discussed, 4 ruled table chatter, 2 rejected.
2. **864 triaged tokens** from the nine sessions whose review pages enumerated what was left alone: 628 left alone, 152 ruled table chatter, 84 that became approved name-correction cards. “Left alone” means the GM saw the token listed and did not pull it back, a weaker label than a card ruling.

Model	Family	Backbone	Median latency per request
clef	Clef	Qwen3.8-27B	420–960 ms
clef-flash	Clef	Qwen3.5-9B	125–270 ms
Decision-2.0-Lux-9B	Decision 2.0	Qwen3.5-9B	127–295 ms (claimed 18.4 ms)
Decision-2.0-Nox-4B	Decision 2.0	Qwen3.5 (4B)	79–206 ms
Decision-2.0-Kai-0.6B	Decision 2.0	Qwen3-0.6B	17–53 ms (claimed 4.9 ms)
jev-1.13.0	TypeSafe (hosted)	not disclosed	116–121 ms*
qwen3.8-flash-next, fast	generative	—	2.0–4.9 s
qwen3.8-flash-next, thinking	generative	—	[QT-LAT] s

Table 1: Models and measured median per-request latency for the request shapes used here (one request, one to two questions, a few hundred to $\sim 2,000$ input tokens). Ranges span tasks. Local decision models were measured single-stream on the GB10; the generative model ran 8-way (fast) or 6-way (thinking) concurrent. *Jev’s figure is end to end from the workstation, including the internet round trip, and is not comparable to the GB10 rows. Claimed latencies are from the model cards, measured on an unnamed GPU.

3. **497 spelling items:** 485 GM-approved name corrections whose canonical form exists in the campaign’s name lists, plus 12 ruled table chatter (correct answer: leave as spoken).
4. **115 entity-type merge rulings** over ensemble dossiers: 109 merged (with a GM-chosen primary type) and 9 kept separate (some groups mix outcomes).

The original per-experiment scripts rebuilt their inputs from the live campaign tree on every run, so the label sets grew while the study ran (the review cards from 525 to 766). For this paper every input was rendered once, with each model-facing `state` already built, and frozen into a read-only, versioned dataset; every model below was run against those same bytes, and a manifest records each file’s hash and the source commits. Rerunning on the frozen set reproduced the deterministic results of the earlier, unfrozen runs exactly where the inputs had not changed (E4 per-fact typing, the E4 replay, E6 head-only triage). The campaign text is not released: it quotes play sessions verbatim and the table-chatter labels name real people. The evaluation scripts and the manifest are public [10].

3.3 Metrics

For verdict prediction we report ROC AUC [8]. For filters we report, at each threshold, the fraction of items forwarded and the number of real names *missed*, since in this pipeline a filtered-out real name is a silent deletion that nothing downstream can recover. For choice tasks we report accuracy and separately count *confident wrong names*, the error a tired reviewer approves.

4 Experiments and results

4.1 E1: Predicting the GM’s verdict (and how it fooled us)

Task. For each of the 766 review cards, ask “Will the GM approve this proposed correction as written?” (noul). If this worked, the review page could be sorted so the GM looks hardest at the likely rejections.

Result. On the cards as the upstream model wrote them, clef reached AUC 0.799 and Jev 0.807 (the other decision models 0.71–0.82). Rebuilding each card from raw sources only (transcript lines, the second transcription, a canon lookup) raised clef to 0.919.

Card subset	n (not approved)	clef		jev	
		upstream prose	raw sources	upstream prose	raw sources
All cards	766 (104)	0.799	0.919	0.807	0.630
Proposes a fix	643 (16)	0.549	0.722	0.418	0.689
Open question, no fix	123 (88)	0.803	0.439	0.749	0.470
Baseline: “has a fix”	766 (104)	0.897			

Table 2: E1, AUC for predicting the GM’s verdict. The headline AUC comes from card type and from upstream prose, not from judgment about the correction itself. The pattern holds for every decision model tested (open-question AUC 0.73–0.81 with prose, 0.44–0.60 without).

Why it is not a result. A one-feature baseline, “does the card propose a fix at all”, scores 0.897: 88 of the 123 cards with no proposed fix were not approved, against 16 of 643 cards with one. Splitting by card type (Table 2) shows that on open questions the strong score existed only when the classifier could read the upstream model’s prose, and fell *below chance* without it (clef 0.439, Jev 0.470). The classifiers were reading the drafting model’s hedges (“could be X or Y”, “what was said?”) back to us. On proposals, the verdict is saturated (97.5% approved) and the 16 non-approvals are too few to measure.

4.2 E2: Choosing the canonical spelling

Task. Given a transcript token, about 21 cues of raw transcript around it and the second transcription’s reading, pick the canonical spelling from a candidate list: the eight nearest names by spelling and Double Metaphone [9] similarity, plus OTHER (a real name not in the list) and LEAVE (not a misheard name). Candidates came from canonical names only; the glossary’s wrong-form column was excluded because it contains these very rulings.

Candidate recall caps every model. The correct name was among the eight candidates for 385 of 485 items (79%). The misses are mostly cases where neither spelling nor sound is close enough to retrieve the name (*Vira* → *Ilvara*, *Peto* → *Buppido*), plus a house convention of transcribing the GM’s own name as “GM”.

Instruction sensitivity. With the question phrased as “which name was spoken”, both models tried on that wording (clef-flash and the fast generative model) expanded a spoken surname into a full name (*Misrim* → “Ilvara Mizzrym”), which would insert words nobody said. Rephrased as “the correct spelling of exactly the words spoken”, the generative model complied and clef did not.

Result (Table 3). Decision models lose to the generative baseline, and the open models below 27B are not usable for this task. The two strongest decision models reach similar accuracy by different routes. Clef commits: 109 of its 167 errors are a wrong name, the error a reviewer approves. Jev abstains: 119 of its 154 errors are OTHER on an item whose name was in the list, and it answered OTHER correctly for 89 of the 100 items whose name was not. Its confidence is informative: the 79 items it answered with confidence ≥ 0.9 were 92.4% correct with 2 wrong names, a slice that could be pre-filled for the GM. Note the test set’s bias: every item is a proposal the upstream model made and the GM approved, so it measures how close these models come to approved work and cannot reveal the upstream model’s own misses.

Model	Accuracy	Confident wrong names	Said OTHER	Most-confident half	Median latency
Decision-2.0-Kai-0.6B	15.3%	54	—	16.5%	0.05 s
Decision-2.0-Nox-4B	19.7%	29	—	20.2%	0.21 s
clef-flash	30.0%	64	—	32.7%	0.27 s
Decision-2.0-Lux-9B	31.4%	44	—	34.7%	0.30 s
clef	66.4%	109	53	78.6%	0.96 s
jev	69.0%	28	208	83.1%	0.12 s*
qwen, fast	76.1%	87	72	—	4.9 s
qwen, thinking	[QT-ACC]	[QT-WN]	—	—	[QT-LAT] s answered
clef $\wedge$ jev agree	90.3% of 268	13	—	—	—
clef $\wedge$ qwen-thinking agree	[QT-AGREE]	[QT-AGREE-WN]	—	—	—

Table 3: E2, 497 items. “Said OTHER” counts every OTHER answer, right or wrong; the Decision 2.0 models and clef-flash mostly answered LEAVE or OTHER instead of reading the candidates. “Most-confident half” is the accuracy of the half of items with the highest reported confidence; the fast generative model’s stated confidence did not separate them. [QT-NOTE]

Model	AUC	Fwd., 0 of 84 missed	Fwd., ≤ 2 missed	Held-out halves (misses)	Chatter recognised	Median
clef	0.933	34–42%	27–39%	43% (0), 49% (0)	136/152	419 ms
jev	0.931	42–55%	37–50%	51% (0), 47% (0)	143/152	116 ms*
clef-flash	0.904	92–95%	47–58%	90% (1), 100% (0)	128/152	124 ms
Decision-2.0-Lux-9B	0.896	75–89%	33–49%	69% (1), 96% (0)	129/152	127 ms
Decision-2.0-Nox-4B	0.900	82–89%	34–46%	96% (0), 94% (0)	118/152	79 ms
Decision-2.0-Kai-0.6B	0.796	97–98%	49–55%	97% (0), 100% (0)	7/152	17 ms
qwen, fast	—	—	24% (6 missed)	—	145/152	2.0 s

Table 4: E3, 864 tokens, 84 of them real names. “Fwd.” is the share of tokens sent on to the expensive step at the threshold achieving the stated number of misses; ranges span two readouts of the same answer ($P(\text{campaign})$ and $1 - P(\text{filler}) - P(\text{realworld})$). “Held-out halves”: a threshold set at half the lowest name score on one half of the sessions, applied to the other half.

4.3 E3: Triage of the unknown-token pile

Task. Before any expensive judgment, classify each capitalised unknown token as filler, rules term, real-world chatter, or campaign name (**choice**), and forward it only if $P(\text{campaign})$ clears a threshold. Nothing is deleted: tokens below the threshold still appear in the review page’s “left alone” list.

Result (Table 4). This is the task where decision models earned a place. Clef 27B and Jev each roughly halve the expensive step’s input without a single silent loss, and both survive a held-out split; Jev also recognised the most table chatter of any decision model. Smaller open models trade that guarantee for speed; the names they drop recur across models (e.g. *Heel Strike*).

4.4 E4: Entity typing in ensemble extraction

Per entity. For each of the 92 usable merged groups, the classifier saw a sample of the entity’s raw facts with the original type labels hidden. A free majority vote over the extraction passes’ own types matched the GM’s primary on 82/92; the decision models matched 79–82 (clef and Nox 82, Jev 80) and the fast generative model 80. Voting with the model deciding only when no type holds 60% of facts scored 85–87 for every model (Jev 86). Most disagreements are filing conventions that the facts do not reveal: the GM filed the demon lords Zuggtmoy, Juiblex and Demogorgon as *monster* but Yeenoghu, also a demon lord, as *npc*; the campaign’s own entity registry types all four *npc*. These are canon questions for the GM, not model errors.

Per fact. The idea tested was to let the generative model extract facts and a decision model

Who types each fact	Per-fact accuracy	Split entities (of 92 merged)	New splits (of 54 consistent)	Facts moved to non-entity types
extraction (as is)	84.3%	78	0 (by selection)	—
clef	76.8%	29	7	255 + 177
jev	84.2%	35	9	87 + 80
clef-flash	81.6%	46	13	120 + 68
Decision-2.0-Lux-9B	86.3%	41	12	22 + 22
Decision-2.0-Nox-4B	83.6%	44	12	42 + 28
Decision-2.0-Kai-0.6B	63.6%	65	34	200 + 137
qwen, fast	88.3%	38	12	28 + 17

Table 5: E4, per-fact re-typing. Accuracy is on the repair set against the GM’s primary type. “Facts moved” gives repair set + control set. Projected over the campaign’s ~ 470 consistent dossiers, re-typing creates roughly as many splits as it repairs (clef, Jev) or more (the others).

	Per-fact types (today)	Vote only, $\geq 60\%$	GM rulings + vote
Named subjects split across types (of 50)	43	9	1 [†]
GM groups in one dossier under the GM’s primary (of 41)	0	32	40
GM groups in one dossier under the wrong type	1	1	0
Named subjects sent to the GM queue	—	9	0
Dossiers	667	592	577

Table 6: E4, entity-level typing replayed on the *monster* slice. [†]The remaining split is one the GM ruled: a group whose *npc* and *monster* files were merged while its *faction* file was kept separate.

assign each fact’s type. On the 1,691 facts of those entities, extraction had typed 266 with a different entity type, splitting 78 entities. Re-typing reduced splits (Table 5), but this set was *selected on extraction’s mistakes*, so any second opinion looks good on it. On a control set of 54 entities that extraction had typed consistently (839 facts), every re-typer created new splits, and clef moved 21% of facts into non-entity types (*event*, *thread*, *date*), removing them from their dossiers.

As an extra signal. Adding decision-model votes to the majority vote hurt (81/92 voting over clef’s per-fact types, 78/92 over Jev’s; 82–84/92 pooled with extraction’s; 84/92 for extraction alone on this fact set). Used instead as a *second opinion* that flags subjects where it disagrees with the vote, clef flagged 15/92 and caught 8 of the vote’s 10 errors (7 false alarms); Jev (15 flags, 7/10), the fast generative model (17, 8/10) and clef-flash (18, 8/10) did about as well.

Interpretation. The failure is structural. Per-fact typing gives an entity with dozens of facts dozens of independent chances to draw one odd label, and one odd label is a split. A deterministic model guarantees the same answer for the same input, but every fact is a different input. The repair is to type once per entity, which needs no model.

Replaying entity-level typing. We then specified that repair as a resolution chain run before dossiers are built: (1) GM filing conventions, (2) the GM’s prior merge rulings, (3) the entity registry’s type, (4) a majority vote over the subject’s own facts if one type holds at least 60%, otherwise (5) a GM review queue. No step calls a model. We replayed it, read-only, against the real bundling rules on the 438 subjects of the campaign’s extraction corpus that have at least one *monster*-typed fact (50 of them named), scoring against the 41 GM merge rulings whose files fall in that corpus (Table 6). The informative arm is the vote alone, because it uses no GM data: it is what a new, unreviewed chapter gets. When it decides, it matches the GM’s primary type 32 times in 33; the one miss is a subject the GM had already flagged as mis-attributed. What it declines to decide is the right set to show the GM: the nine queued named subjects are mostly the demon-lord, deity and place-versus-organisation questions above. A threshold sweep put the knee at 60%: 50% adds three wrong types, 67% more than doubles the queue. Adding the GM’s merge rulings reproduces all 40 merged groups that fell in the slice (partly circular, since it is scored against those rulings).

Policy	Pairs moving the expected way ($\Delta > 0.02$)	Goblin $P(\text{flee})$: unhurt $\rightarrow$ alone $\rightarrow$ cornered
FlexAI d100 tables (role $\times$ stance)	5 / 12	0.15 $\rightarrow$ 0.16 $\rightarrow$ 0.00
clef	11 / 12	0.03 $\rightarrow$ 0.75 $\rightarrow$ 0.43
clef-flash	11 / 12	0.03 $\rightarrow$ 0.48 $\rightarrow$ 0.09
Decision-2.0-Lux-9B	11 / 12	0.01 $\rightarrow$ 0.59 $\rightarrow$ 0.17
Decision-2.0-Nox-4B	11 / 12	0.01 $\rightarrow$ 0.49 $\rightarrow$ 0.10
Decision-2.0-Kai-0.6B	5 / 12	0.05 $\rightarrow$ 0.16 $\rightarrow$ 0.20
jev	11 / 12	0.00 $\rightarrow$ 0.89 $\rightarrow$ 0.09
qwen, fast (asked for a distribution)	9 / 12	0.00 $\rightarrow$ 0.70 $\rightarrow$ 0.00

Table 7: E5. Two pairs are imperfect: in one, clef’s answer (an archer engaged in melee draws its sword, *attack secondary* 0.64) is arguably better than the expected *maneuver*; in another, the shared party description contradicted the scenario’s premise, invalidating the pair for all policies. The scenarios and expectations were written by the experimenter.

The replay also surfaced two defects in the specification and one in the pipeline. Keep-separate rulings must apply to a *file* (type and subject), not a subject, because a single group can be partly merged and partly kept apart. The review queue must hold named subjects only; all 20 subjects queued once the GM’s rulings were applied were generic creatures. And subjects are canonicalised only through the registry’s aliases, so misspellings known to the spelling glossary but not the registry, and extractions made before a source-text correction, split subjects before typing ever runs. The underlying flaw is general: the pipeline asks the generative model for precision decisions (a fact’s type decides its scope, its subject decides its identity) and then stores the GM’s corrections where the pipeline never reads them, so every run rebuilds what the GM already fixed.

4.5 E5: NPC behaviour

Task. Twelve paired combat scenarios, each pair differing in one fact, with the expected direction stated before running (e.g. a goblin with all allies dead and an open tunnel behind it should flee more; the same goblin in a dead-end alcove should flee less; a mindless zombie should not flee). Two questions per request in one call: what the creature does (FlexAI’s six outcomes) and whom it targets (FlexAI’s eight targeting rules). FlexAI’s own d100 tables are the baseline; they see only role and stance.

Result (Table 7). Situation-aware decision models clearly beat the dice tables, and their probabilities are graded, which is what a sampler needs. They differ in how far they move: clef keeps a cornered goblin at 0.43 *flee*, Jev drops it to 0.09 after putting 0.89 on fleeing when the goblin is alone with an open tunnel. Which is right is a table-style question, not one the pairs can settle. The generative model’s stated distributions were nearly degenerate (many 0.00 and 1.00 values, e.g. 1.00 on a single target), so sampling from them adds little variety. On this evidence we built a table-side prototype: the GM types one line describing the situation, a decision model returns stance probabilities over eight social responses and samples one, and the generative model voices a line in that stance. With Decision-2.0-Nox-4B a suggestion takes 0.25–0.8 s including the NPC’s dossier, and the line about 2.5–3 s. A labelled evaluation of social responses against 60 past moments has been prepared but not yet adjudicated by the GM; we do not report it.

4.6 E6: Fine-tuning a decision model on the GM’s rulings

Question. Can the failures above be trained away? We fine-tuned Decision-2.0-Nox-4B on the GM’s rulings for two tasks: triage (E3) and per-fact entity typing (E4, where the labels encode the GM’s filing conventions). Jev cannot be tried here: TypeSafe does not fine-tune it per customer

Nox-4B	Triage (864 tokens)			Entity typing (2,530 facts, 146 entities)		
	AUC	Fwd. @ 0 miss	Fwd. @ ≤ 2	Per-fact acc.	Split (92 / 54)	Entity majority
released	0.900	82%	34%	87.2%	44 / 12	[ENT-REL] / 146
head only	0.963	67%	34%	91.3%	34 / 9	—
LoRA r16 + head	0.935	83%	36%	[LT-ACC]	[LT-SPLIT]	[ENT-LORA] / 146

Table 8: E6, held-out results. “Fwd.” is the share of tokens forwarded at the best held-out threshold for the stated number of missed names. “Split” counts entities whose facts land in more than one entity type (repair set / control set). Head-only settings were chosen from four candidates on the same cross-validation, so its rows are mildly optimistic. LoRA training is not bit-reproducible; an earlier run on the unfrozen inputs gave triage AUC 0.915.

and serves the same weights to every account [1].

Method. The released Transformers wrapper refuses `train()`, but the runtime’s native backbone and head are ordinary PyTorch modules. We loaded the model through the runtime exactly as the server does and confirmed that cached features reproduced the served probabilities (mean $|\Delta p| = 0.0009$). We then trained in two stages: *head only* on cached features, and a *rank-16 LoRA* on the MLP and attention projections plus the head, with gradients through the backbone. The loss is valid- k cross-entropy ($-\log \sum_{k \in \text{valid}} p_k$, the loss family the release reports), with an L2 pull toward the released head. Evaluation is cross-validated by *session* for triage (3 folds) and by *entity* for typing (5 folds), so no test entity or session is seen in training.

Result (Table 8). Head-only tuning *recalibrates* the readout. Triage AUC rose from 0.900 to 0.963, and per-fact typing accuracy from 87.2% to 91.3%, with almost no facts drifting into non-entity types (42 + 28 to 0 + 1). It did not learn the conventions. The demon lords the GM files as *monster* moved *toward npc* (Demogorgon: 12 of 12 facts *monster* released, 1 of 12 tuned; Juiblex: 7 of 14 to 0), outvoted by a label distribution that is 64% *npc* and by the GM’s own contrary ruling on Yeenoghu. In the earlier, unfrozen run, weak regularisation overfit badly: a threshold chosen on the training sessions missed 19 held-out names. [LORA-RESULT] Three triage names (*Heel Strike*, *Zoom*, *Pick / Shine*) were missed by every model and every tuning, which suggests a limit of the label given this input rather than of model capacity.

4.7 E7: Serving on the GB10

Latency. Table 1 summarises it. Decision 2.0’s runtime ships custom kernels only for AMD’s gfx942; on CUDA it uses pure-PyTorch reference kernels, and its CUDA path is marked unvalidated upstream. Measured medians on the GB10 were 3–7 $\times$ the model-card figures (Kai 17 vs. 4.9 ms; Lux 127 vs. 18.4 ms). For Clef, the GB10 was 3.5 $\times$ slower than Cloudflare’s published H200 medians for the 9B model but only 1.9 $\times$ for the 27B, consistent with fixed per-request overhead dominating small models. Hosted Jev answered in 119 ms at the median and 232 ms at the 99th percentile, internet round trip included; a few requests stalled for tens of seconds in rate-limit back-off. The whole Jev run (6,292 requests, 4.2M input tokens) cost about \$0.18 at the listed price.

Correctness. With no upstream golden check on CUDA, we compared Kai-0.6B’s CUDA and CPU outputs on identical requests: probabilities agreed within 0.005.

Memory. The runtime’s placement check used `torch.cuda.mem_get_info`, which on the GB10’s unified memory excludes reclaimable page cache; the box reported 9–18 GiB “free” with 34 GB available, and reading the weights to verify checksums refilled the cache just before the check. We patched the check to use the kernel’s `MemAvailable` on integrated devices. Separately, Lux-9B occupied about 50 GB (33 GB resident in the process) for 16 GB of BF16 weights; co-hosted with

Clef the system fell to 1 GB available and the runtime crashed once. For the frozen-dataset rerun each family was therefore served alone, one after another.

5 Discussion

What kind of label is it? Across the five tasks a single distinction predicted success. Decision models succeeded when the label is a *coarse property readable from meaning*: is this token chatter or a campaign name; would a cornered creature flee. They fell short when the label is (a) a *string-level* judgment (which spelling, exactly which words), (b) a *convention* the text does not reveal (how this GM files demon lords), or (c) a *consistency property across items* (one type per entity across many facts). Most judgment steps in this pipeline are of the second kind, which is why the decision models' role ended up narrow. Fine-tuning (E6) did not move this boundary: training on a few thousand of the GM's own rulings sharpened the readout on clear cases, but a sparse, partly contradictory convention was outvoted rather than learned. Jev, a closed model trained by its vendor for calibrated decisions, sits on the same side of every boundary as the open models; it differs in how it fails.

Abstaining is the property to ask for. On spelling, clef and Jev were equally accurate, but clef's errors were mostly wrong names and Jev's were mostly abstentions, and Jev's confidence separated right from wrong more sharply (its answers at confidence ≥ 0.9 : 79 items, 92.4% correct; clef's: 20 items, 70%). For a pipeline with a human checkpoint that difference is decisive: an abstention costs the GM a lookup, a confident wrong name costs a silent error in the record. Accuracy tables hide it; we recommend counting the two kinds of error separately for any decision model placed in front of a reviewer.

Filters and flags, not deciders. Every useful configuration kept the human checkpoint intact: triage that orders work without deleting it, a second opinion that adds subjects to a review queue, and sampled NPC suggestions the GM reads. None of these removes a decision from the human.

A ruling should be made once. The checkpoint only pays if its output flows back in. The ensemble pipeline asked a generative model for per-fact precision decisions and stored the GM's 109 corrective merges where no stage read them, so each run recreated the same splits for the GM to fix again. Resolving each precision field against the GM's accumulated rulings, with a deterministic default and a queue for the rest, did more for consistency than any classifier we tried (E4 replay). The better model was not the missing piece; the missing piece was reading the rulings back.

Calibration has two uses. Graded probabilities mattered in two different ways: to set a zero-loss filter threshold (E3) and to sample plausible behaviour (E5). The generative baseline's self-reported probabilities served neither purpose.

Local versus hosted. On the two tasks where decision models earned a place, the hosted Jev matched the best local model (clef 27B) on triage and every 4B-and-larger local model on NPC behaviour, at a fraction of the latency of clef 27B. The local path's value here was not quality: it was the ability to fine-tune (E6), to inspect the head, and to find the GB10-specific failures (E7).

Two ways a convincing result was wrong. E1's AUC came from the classifier reading the upstream model's prose, and E4's per-fact improvement came from a test set selected on errors. Both looked strong until a baseline or a control was added. We recommend both checks for any

evaluation built from a pipeline’s own review records, and freezing the inputs before comparing models: our own label sets grew by half between the first runs and this paper.

6 Limitations

This is a single-practitioner study: one campaign, one GM, one hardware platform. Several label sets are small (84 real names in E3; 16 non-approved proposals in E1; 12 scenario pairs in E5). Some labels are weak (“left alone” in E3), and some test material was written by the experimenter (E5). The E2 set was built from approved proposals and cannot reveal the upstream model’s misses. Prompts and option descriptions were written once and not tuned, which is fair to a drop-in classifier but unfavourable to it; TypeSafe’s own guidance favours structured instructions and criteria, which we did not use, so Jev in particular may be under-served. Jev is a hosted model of undisclosed size and training data, called at one pinned version; its results describe that version, and its latency includes our internet path. Running it sent the frozen inputs, which include transcript excerpts, to a third-party service. We did not verify that the locally served Clef matches Cloudflare’s hosted service. Decision 2.0 ran on a runtime path its authors mark as unvalidated on CUDA; our own CPU/GPU parity check covers only the smallest model. Local latencies are single-stream; the runtime’s batching profiles were not tested. Fine-tuning explored one LoRA configuration (rank 16, one or two epochs) and selected head-only settings on the same cross-validation folds. The entity-level typing replay covers only subjects with a *monster*-typed fact in one of two extraction corpora, and 41 of the 64 relevant GM rulings; the other entity types are not yet replayed.

7 Conclusion

Decision models are a useful addition to a local LLM toolkit, in a narrow role: fast, calibrated classification of coarse properties, placed where it orders or flags work rather than deciding it. In this pipeline that meant triaging transcript tokens without silent losses and generating situation-aware NPC behaviour at the table. They did not replace a generative model for spelling decisions, could not infer house conventions, and cannot impose consistency across items that are judged one at a time; that consistency came instead from typing once per entity and feeding the GM’s own rulings back into the pipeline. Where they fall short, the useful question is whether a model abstains or guesses: the hosted Jev mostly abstained and the open models mostly guessed. On the GB10, the smaller Decision 2.0 models give most of the behavioural quality at a fraction of the memory, but published latency claims should be re-measured on the target hardware.

Data and code availability

Serving configurations, the GB10 patch, all evaluation scripts, and the frozen dataset’s manifest (file hashes, counts and source commits) are available in the public repository [10]. The campaign data itself is not released because it quotes play sessions verbatim and names real people.

Use of AI assistance

The experiments, analysis and a first draft of this paper were carried out by an AI coding agent (Claude, Anthropic) working under the author’s direction, with the author reviewing results and making the decisions recorded in the paper. The author takes full responsibility for the content.

References

- [1] TypeSafe. Jev and the System One API. Documentation, <https://docs.typesafe.ai>, model `jev-1.13.0`, accessed October 2026.
- [2] Cloudflare. Clef and Clef-flash decision models. Model cards, <https://huggingface.co/Cloudflare/clef> and <https://huggingface.co/Cloudflare/clef-flash>, October 2026.
- [3] vLLM Semantic Router Team. Decision 2.0: towards open foundation decision models. Model collection, <https://huggingface.co/collections/vllm-sr/decision-20>, October 2026.
- [4] vLLM Semantic Router Team. Built-in model runtime, phase 1: serve Decision 2.0 from vllm-sr. GitHub issue #4479, <https://github.com/vllm-project/semantic-router/issues/4479>, 2026.
- [5] vLLM Semantic Router. When to reason: Semantic router for vLLM. arXiv:2510.08731, 2025.
- [6] vLLM Semantic Router. The workload-router-pool architecture for LLM inference optimization: A vision paper from the vLLM Semantic Router project. arXiv:2603.21354, 2026.
- [7] Infinium Game Studios. *FlexAI Guidebook*. 2020.
- [8] J. A. Hanley and B. J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982.
- [9] L. Philips. The double metaphone search algorithm. *C/C++ Users Journal*, 18(6):38–43, 2000.
- [10] K. Roussos. dgx-fun: experiments with local LLM serving on NVIDIA DGX Spark. <https://github.com/kostadis/dgx-fun>, 2026.